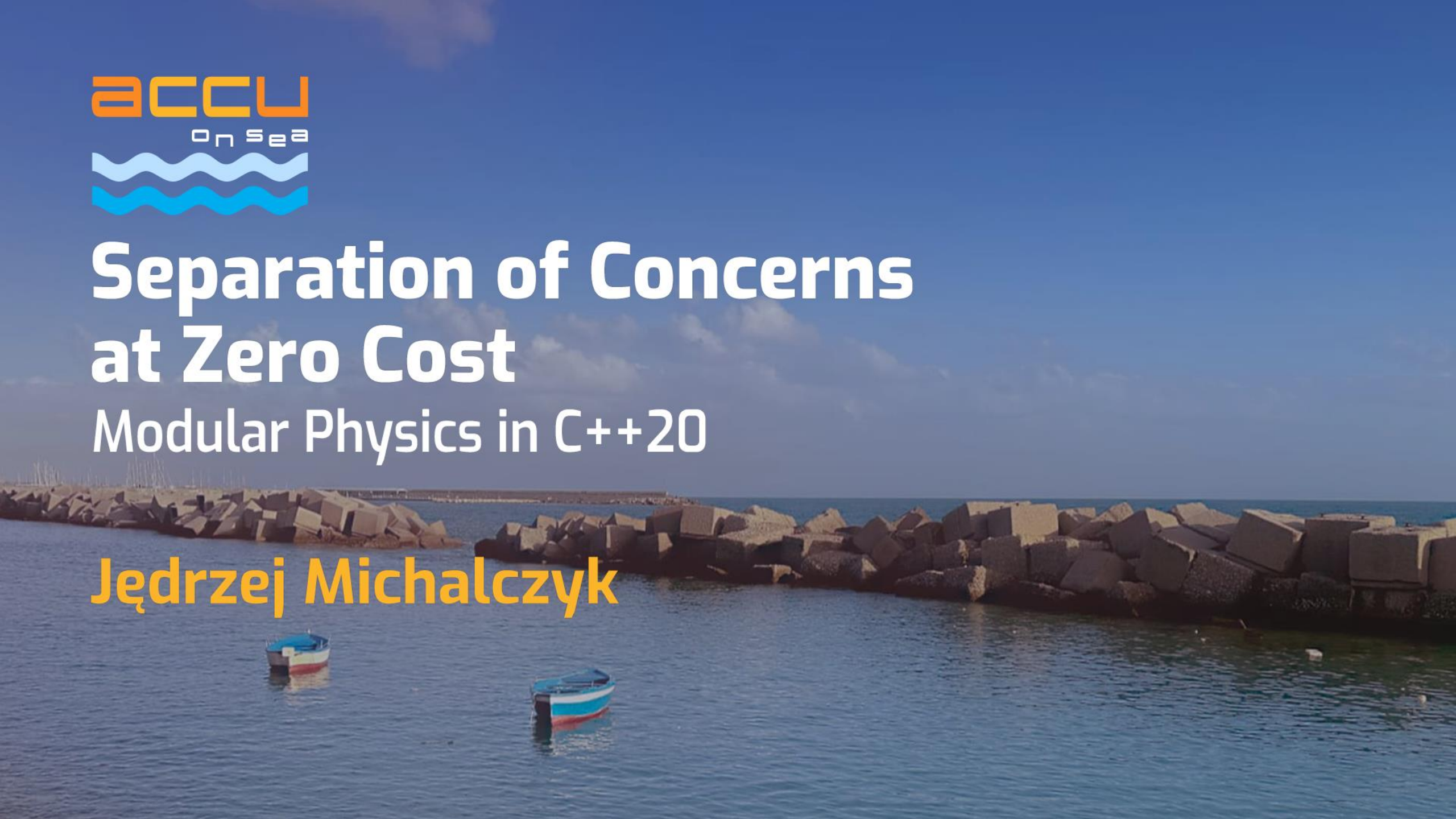


Separation of Concerns at Zero Cost

Modular Physics in C++20

Jędrzej Michalczyk

BEFORE WE START

$ whoami

- **Jędrzej Michalczyk** — flight software & simulation engineer
- **SpaceForest** (Gdynia, Poland) — suborbital rockets
  - PERUN — single-stage suborbital launch vehicle
  - guidance, navigation & control software in modern C++
- The code in this talk is real: github.com/jedrzejmichalczyk/sopot

[PHOTO: you / PERUN launch]

SPACEFOREST, A FEW YEARS AGO

The original problem

- Simulate a rocket: **6-DOF dynamics + aerodynamics + propulsion + atmosphere + control**
- The same physics must run in three places:
  - engineering analysis (parameter studies, Monte Carlo)
  - hardware-in-the-loop test benches — real time
  - onboard — guidance runs against the same models, in real time
- **Correctness and performance are both non-negotiable**
- And: aero engineers, control engineers and software engineers all touch the same code

HOW WE GOT HERE

We wrote the same equations three times

1. Mathematica

+ symbolic, trusted, readable by physicists

− cannot fly:

not deployable, slow

2. Hand-crafted C

+ fast, deployable

− write-once, read-never;

every change ripples

through one giant file

3. Generated code

+ fast + traceable to the math

− now the generator

is the unmaintainable

program

The physics never changed. We rewrote it three times.

01

The Principle

Separation of concerns in numerical code

≈ 5 min

A spring shouldn't know how a mass stores its position.

An energy calculator shouldn't care which integrator you use.

Easy to state. Hard to achieve in performance-critical C++.

- Single responsibility → testable in isolation
- Typed interfaces → wrong wiring fails at compile time
- Independent development → aero people don't merge-conflict with control people

*"There are two ways of constructing a software design: one way is to make it **so simple that there are obviously no deficiencies**, and the other way is to make it so complicated that there are no obvious deficiencies."*

— C.A.R. Hoare, The Emperor's Old Clothes, Turing Award lecture (1980)

SOPOT — "Obviously Physics, Obviously Templates" — picks the first way.

THE STATUS QUO

What numerical code usually looks like

```
// derivs.cpp — all of physics in one function
void derivs(double t, const double* y, double* dydt) {
    // y[0..2] pos, y[3..5] vel, y[6..9] quat, y[10..12] omega, y[13] mass
    double rho = 1.225 * exp(-y[2] / 8500.0);            // atmosphere
    double v2  = y[3]*y[3] + y[4]*y[4] + y[5]*y[5];
    double Fd  = 0.5 * rho * v2 * CD * AREA;             // aero drag
    double Ft  = thrust_table_lookup(t);                 // propulsion
    dydt[3] = (Ft*ax(y) - Fd*vx_hat(y)) / y[13];         // dynamics
    /* ... 400 more lines, every subsystem interleaved ... */
}
```

- **Indices by convention** — y[2] is altitude… everywhere
- Nothing testable in isolation
- Every change touches the same function
- **But: it's fast**

02

The Tradeoff That Isn't

Modularity vs performance — or so we're told

≈ 7 min

THE TEXTBOOK FIX

Attempt: classic OO interfaces

```
struct Component {
    virtual void computeDerivatives(double t,
                                    const double* state,
                                    double* dydt) = 0;
    virtual double getOutput(const std::string& name) = 0;  // !!
    virtual ~Component() = default;
};

std::vector<std::unique_ptr<Component>> components;
for (auto& c : components)              // innermost loop of RK4,
    c->computeDerivatives(t, y, dydt);  // 4x per step, 1000s of steps
```

- **Virtual dispatch in the hot loop**
- No inlining across components
- Everything is double* — type safety gone
- getOutput("mass1_pos") — stringly typed wiring

PICK ONE?

The apparent tradeoff

Monolith

fast ✓
unmaintainable ✗

Virtual components

modular ✓
indirection in the hot loop ✗

C++20 lets us refuse this choice.

03

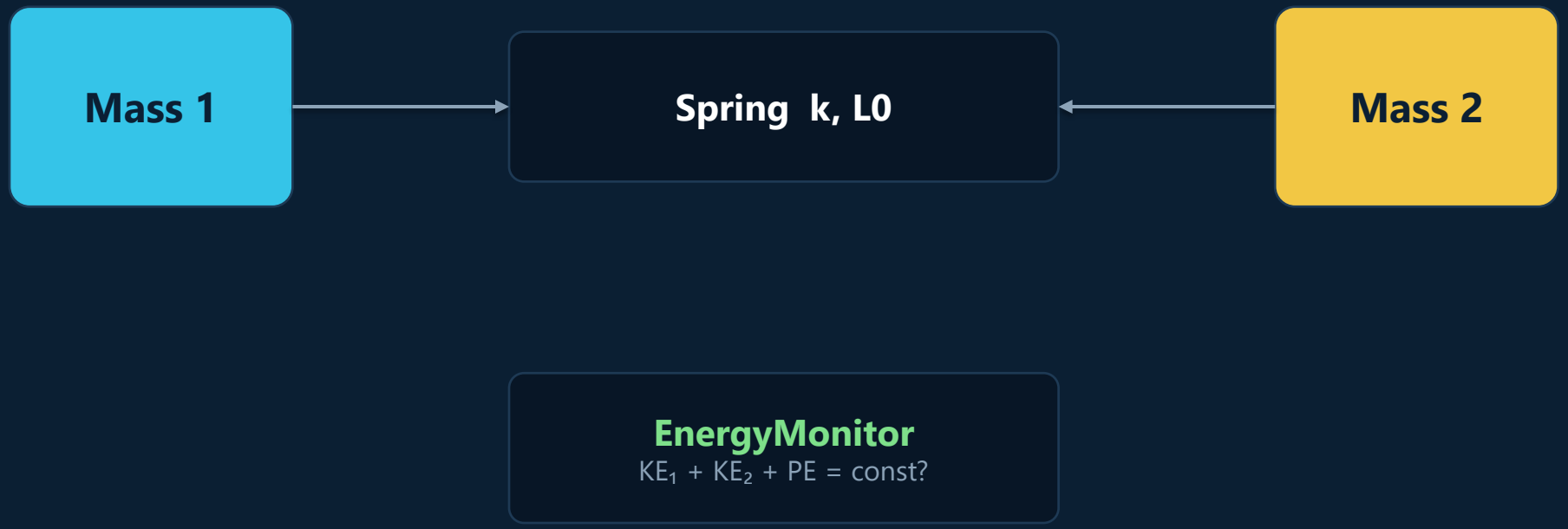
The Coupled Oscillator

Two masses, one spring, four components

≈ 8 min

THE RUNNING EXAMPLE

Small enough for slides, real enough to have a conservation law



- 4 components: **Mass1, Mass2, Spring, EnergyMonitor**
- Each owns its data, queries the rest through **typed tags**
- Total energy gives us a built-in correctness test

SEPARATION, CONCRETELY

Who owns what — who asks what

Component	Owns (state)	Provides	Queries
Mass1 / Mass2	[position, velocity]	Position, Velocity, Mass	Force (whoever provides it)
Spring	— nothing	Force, Extension, PE	Position of both masses
EnergyMonitor	— nothing	TotalEnergy, Momentum	everything above

- The Spring never includes the mass header — only the tag definitions
- The EnergyMonitor doesn't know a spring exists — it asks for spring::PotentialEnergy by tag
- Stateless components (Component<0,T>) are pure functions over the system state

04

Code Walkthrough

The four components, for real

≈ 12 min

WALKTHROUGH 1/5

Tags: typed names for physical quantities

```
struct mass1 {
    struct Position {};      // empty types — pure compile-time keys
    struct Velocity {};
    struct Force    {};
    struct Mass     {};
};

struct mass2 { /* same shape, different types */ };

namespace spring {
    struct Extension       {};
    struct PotentialEnergy {};
};
```

- **A tag is a type** — not a string, not an index
- mass1::Position ≠ mass2::Position at compile time
- Mixing them up is a type error, not a NaN three weeks later

WALKTHROUGH 2/5

PointMass: owns its state, asks for force

```
template<typename TagSet, Scalar T = double>
class PointMass final : public Component<2, T> {   // 2 states: [x, v]
    double m_mass;
public:
    template<typename Registry>
    LocalDerivative derivatives(T t, std::span<const T> state,
                                const Registry& registry) const {
        T velocity = this->localState(state, 1);          // dx/dt = v
        T force = query<typename TagSet::Force>(registry, state);
        return {velocity, force / T(m_mass)};             // dv/dt = F/m
    }

    template<typename Registry>
    T compute(typename TagSet::Position, std::span<const T> s,
              const Registry&) const { return this->localState(s, 0); }
    template<typename Registry>
    T compute(typename TagSet::Velocity, std::span<const T> s,
              const Registry&) const { return this->localState(s, 1); }
};
```

- **Newton's law and nothing else**
- Doesn't know what produces the force
- compute(Tag, ...) overloads = what it offers to others
- Same class is Mass1 or Mass2 via the TagSet parameter

Spring: no state, just physics

```
template<typename TagSet1, typename TagSet2, Scalar T = double>
class Spring final : public Component<0, T> {        // 0 states!
    double m_stiffness, m_rest_length;
public:
    template<typename Registry>
    T compute(typename TagSet1::Force,
              std::span<const T> state, const Registry& registry) const {
        T x1 = query<typename TagSet1::Position>(registry, state);
        T x2 = query<typename TagSet2::Position>(registry, state);
        T extension = x1 - x2 - T(m_rest_length);
        return -T(m_stiffness) * extension;              // Hooke's law
    }

    template<typename Registry>
    T compute(typename TagSet2::Force,
              std::span<const T> state, const Registry& registry) const {
        return -query<typename TagSet1::Force>(registry, state);  // Newton's 3rd
    }
};
```

- **Queries positions, provides forces**
- Doesn't store, doesn't integrate, doesn't allocate
- Never includes the mass header
- Newton's 3rd law is one line

WALKTHROUGH 4/5

EnergyMonitor: system totals from component data

```
template<Scalar T = double>
class EnergyMonitor final : public Component<0, T> {
public:
    template<typename Registry>
    T compute(system::TotalEnergy,
              std::span<const T> state, const Registry& registry) const {
        T m1 = query<mass1::Mass>(registry, state);
        T v1 = query<mass1::Velocity>(registry, state);
        T m2 = query<mass2::Mass>(registry, state);
        T v2 = query<mass2::Velocity>(registry, state);
        T pe = query<spring::PotentialEnergy>(registry, state);
        return T(0.5)*m1*v1*v1 + T(0.5)*m2*v2*v2 + pe;
    }
};
```

- **Asks for quantities by tag**
- Has no idea a spring provides the PE
- Swap the spring for gravity — this file doesn't change

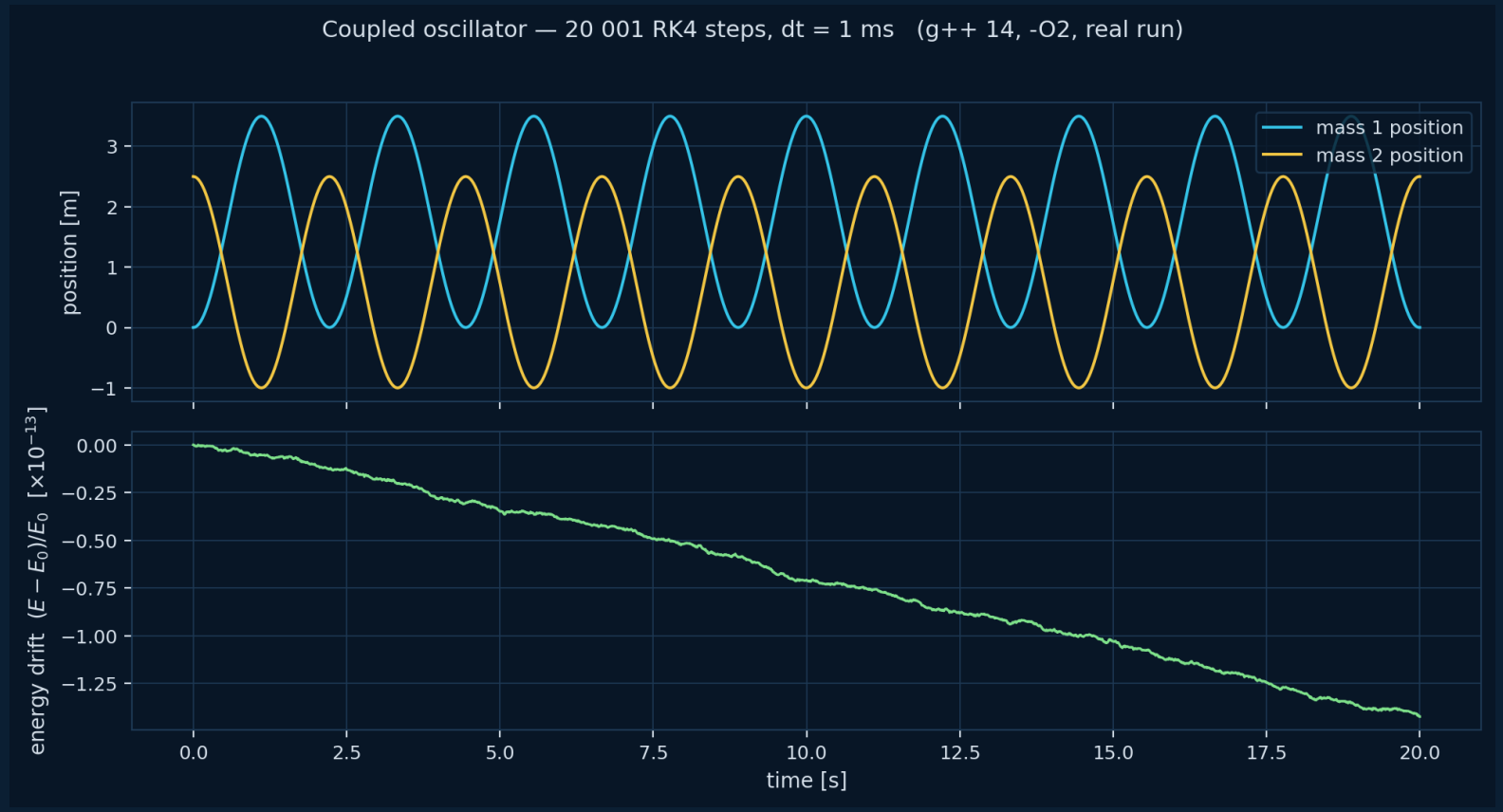
Wiring it together

```
auto sys = makeODESystem<double>(
    Mass1<double>(1.0, /*x0*/ 0.0, /*v0*/ 0.0),
    Mass2<double>(1.0, /*x0*/ 2.5, /*v0*/ 0.0),
    Spring12<double>(/*k*/ 4.0, /*L0*/ 1.0),
    EnergyMonitor<double>()
);

// the wiring is CHECKED at compile time:
static_assert(decltype(sys)::hasFunction<system::TotalEnergy>());

RK4Solver solver;
auto sol = solver.solve(derivs(sys), sys.getStateDimension(),
                        0.0, 20.0, /*dt*/ 0.001, sys.getInitialState());

double E = sys.query<system::TotalEnergy>(sol.states.back());
```

- **Components are values in a tuple — no heap, no pointers**
- Forget the spring → **static_assert fires** — not a runtime surprise

PROOF, NOT PROMISE

Does it work? Ran this morning on my laptop



20 001
RK4 steps (dt = 1 ms)

16.7 ms
wall clock, g++ 14 -O2

1.4×10^{-13}
max relative energy drift over 20 s

That drift is RK4 truncation error — the architecture adds nothing.

05

How It Works

Tags, registry, concepts — no magic

≈ 10 min

Idea 1 — dispatch on type, not through a hierarchy

```
// instead of:                          // we write:
double getOutput(                       T compute(mass1::Position,
    const std::string& name);               std::span<const T> state) const;

// one virtual + string compare         // one overload per quantity,
// per query, at runtime                // chosen by the COMPILER
```

- **Overload resolution replaces virtual dispatch**
- The tag argument is empty — it exists only to select the overload
- Unknown tag → compile error, with the tag's name in the message
- This is ordinary C++ — tag dispatch is as old as the STL (std::advance)

Idea 2 — the registry: 'who provides what', resolved at compile

```
template<typename T, ComponentConcept... Components>
class Registry {
    std::tuple<const Components&...> m_components;

    template<StateTagConcept Tag>
    static constexpr bool hasFunction() {
        return (ProvidesStateFunction<Components, Tag, T, Registry> || ...);
    }                                    //        fold over all components

    template<StateTagConcept Tag>
    auto computeFunction(std::span<const T> state) const {
        static_assert(hasFunction<Tag>(),
                      "No component provides this state function");
        const auto& provider = findProvider<Tag>();   // index found constexpr
        return provider.compute(Tag{}, state, *this);
    }
};
```

- **A fold expression over the component pack**
- findProvider = constexpr search of the tuple
- **After inlining, the 'lookup' is gone**

Idea 3 — concepts make the contracts checkable

```cpp
// what a component IS:
template<typename C>
concept ComponentConcept = requires(const C& c) {
    { C::state_size } -> std::convertible_to<size_t>;
    { c.getInitialLocalState() } -> std::same_as<typename C::LocalState>;
};

// what it PROVIDES (a quantity, selected by tag):
template<typename C, typename Tag, typename T, typename Registry>
concept ProvidesStateFunction =
    ComponentConcept<C> && StateTagConcept<Tag> &&
    requires(const C& c, std::span<const T> s, const Registry& reg)
    { c.compute(Tag{}, s, reg); };
```

- **Errors appear at the interface** — not 40 frames deep in template guts
- The 'component checklist' from our docs is enforced, not advised
- This is what makes the pattern **practical** — pre-C++20 it was an expert-only SFINAE swamp

Idea 4 — everything is a value

- The system is std::tuple<Mass1, Mass2, Spring, EnergyMonitor>
  - no heap, no pointers, no type erasure, no std::function
- **The optimizer sees the whole object graph as one struct**
  - every compute() call is a direct (and inlinable) call
- State lives in one contiguous vector; components know their offsets
- The price: component set is fixed at compile time
  - for flight software that's a feature — the vehicle doesn't grow a fin mid-flight

06

What the Compiler Sees

Trust, but disassemble

≈ 6 min

G++ 14 -O2, REAL OUTPUT

The abstraction compiles away

```
energy_sopot(span<const double>):     ; 4 components, registry dispatch
    mov   r10, [sys+272]
    movsd xmm1, [.LC0]          ; 0.5
    mov   rax, [rcx]
    movsd xmm0, [r10+8]         ; m1        <- runtime parameters,
    movsd xmm3, [rax+rcx*8]     ; v1           loaded once
    mulsd xmm0, xmm1
    mulsd xmm0, xmm3            ; 0.5*m1*v1
    mulsd xmm0, xmm3            ; ... * v1
    subsd xmm2, [r8+16]         ; x1-x2-L0
    mulsd xmm1, xmm2            ; PE terms
    addsd xmm0, xmm3
    addsd xmm0, xmm1            ; KE1+KE2+PE
    ret
```

- **0 call 0 jmp 0 vtable**
- Straight-line floating point — identical shape to the hand-written formula
- Only difference: the monolith constant-folds m and k; the framework reads them as parameters
- Registry, tags, concepts: **zero instructions**

MEASURED, NOT ESTIMATED

Numbers

~1 ns per state-function query — fully inlined

16.7 ms 20 001 RK4 steps of the 4-component oscillator (laptop, -O2)

70–80 % of native speed for the same code compiled to WebAssembly

FROM THE HANDWRITTEN PLAN

Why C++20 — and not C++26?

- **Everything here needs only C++20:**
  - concepts (the diagnostics!), fold expressions, std::span, CTAD
- **C++26 reflection (P2996) would remove the remaining boilerplate:**
  - tag declarations and compute() registration could be generated
  - we have working experiments in the repo: reflect/
- But flight software ships on the toolchain you have, **not the one you want**
  - embedded / cross targets lag years behind; certification lags more

07

Where It Scales

From two masses to a flying vehicle

≈ 7 min

PRODUCTION, NOT PROMO

Same pattern, real vehicle: 6-DOF rocket

- **11 components, 14 states** — position, velocity, quaternion, angular rate, mass
- Thrust vector control & attitude determination: same tags-and-registry architecture
- **These patterns run in production flight software** — correctness and performance both non-negotiable

Bonus: the scalar is a template parameter

```
// simulate:
auto sim  = makeODESystem<double>(comps...);

// differentiate — SAME components, different scalar:
using D = Dual<double, 14>;                  // forward-mode autodiff
auto grad = makeODESystem<D>(comps...);
auto J    = computeJacobian(grad, t, state); // exact, to machine precision

// linearize -> LQR gain straight from the nonlinear model
auto K = lqr(J.A, J.B, Q, R);
```

- **Jacobians without hand-derived math or finite differences**
- The controller you'll see in a minute was designed exactly this way

OPEN IDEAS

Can we go further?

- **Frames as types** — body vs ENU vs ECEF: mixing them up should be a compile error too
- **Physical laws as concepts** — 'conserves energy', 'action-reaction pair' as checkable contracts
- Units in the type system — dimensional analysis at compile time

The goal, all the way down: a consistent level of abstraction —
the code reads at the level of the physics, and the machine code reads like the monolith.

A SHORT HISTORY LESSON

We've been here before

[PLACEHOLDER — scan of rocket-control code from a classic textbook
(Zipfel / 1960s–90s Fortran, zero modularity)]

- Six decades of rocket code — the math was always modular; the code never was
- **Not because the engineers were careless — because the languages made them choose**

FUN EXAMPLE — LIVE

One more thing: six pendulums on a cart

- **Inverted 6-pendulum chain** balanced on a moving cart — LQR-stabilized
- Built from the same components, tags and registry you just saw
- Linearized with Dual<double, N> — the autodiff Jacobian from two slides ago
- Same C++20, compiled to WebAssembly, running in your browser right now:

jedrzejmichalczyk.github.io/sopot

▶ LIVE DEMO

IF YOU REMEMBER FOUR THINGS

Takeaways

- **Separation of concerns is achievable at zero cost** in C++20 — measured, not asserted
- The recipe is small: **typed tags + overloaded compute() + a constexpr registry + concepts**
  - no inheritance, no virtuals, no strings, no codegen
- **Prefer abstractions you can verify** — conservation laws and disassembly are unit tests for architecture
- **Quality vs performance was the false tradeoff** — modern C++ finally lets us refuse it

Thank you! Questions?

- Code & slides: **github.com/jedrzejmichalczyk/sopot**
- Live demo: **jedrzejmichalczyk.github.io/sopot**
- Email: **jedrzej.michalczyk@spaceforest.pl**